Technical support for an office network

Multi-agent system with safety rules in code · Alberto Olmos · Laboratory Practice I, Agentic AI · October 2026

1. Problem and scope

The system acts as **first-level support** for the employees of four offices (MAD-01, MAD-02, BCN-01, VAL-01). The input is the incident text, the office and an optional ticket; the output is a **Result** with the final status (*resolved*, *escalated*, *rejected* or *step_limit_reached*), the answer to the employee with citations to the knowledge base, and the full trace. Whatever is not resolved is escalated to the second level with a summary of what was tried. Requests that are not office support (rejected) and any physical repair are out of scope. It is written in Python 3.12 **without agent frameworks**: the loop and the rules are my own code, which I can explain and test.

2. Multi-agent architecture

Agent	Decides	Output (validated JSON)	Tools
classifier	type and priority	type, priority, reason	search_tickets
diagnosis	probable cause	cause, service, evidence	check_service, check_device, search_tickets
knowledge	procedure	procedure, citations, action	search_docs; *proposes* restart_service and close_ticket
escalation	handoff to level 2	summary, tried, priority	create_escalation

Handoff orchestration: classifier → diagnosis → knowledge → resolved or escalated. Support is sequential (a solution cannot be proposed without knowing the cause), so a supervisor would only add model calls and points of failure. The exceptions are fixed rules in code: a reopened ticket, an outage in several offices or a safety hazard → escalation after diagnosis; no valid citation or a rejected action → escalation.

State and messages: a shared `State` keeps what has been found out (type, priority, cause, service, citations, actions, trace and steps). Each agent receives a JSON summary of that state, not the whole conversation: less text, fewer credits and less confusion. In each turn the model returns a single JSON object (a tool call or the final output); the code validates the contract, and the prompt is generated from the registry with **only** the tools allowed to that agent (least privilege).

Priorities. The classifier proposes them; the code **raises** the priority if the data require more (it never lowers it), a safety hazard (explosion, fire, smoke, sparks) is at least P2, and a rejection has no priority. Critical services: vpn, email, erp, files and intranet (the intranet hosts password self-service: if it fails, it is a security problem). Non-critical: printing and guest wifi.

	Criterion	Example in the data	1st response
P1	Affects more than one office, or a critical service is down	vpn down in MAD-01; erp down in MAD-02 and BCN-01	15 min
P2	Affects an entire office, or a critical service is degraded	email degraded in BCN-01	30 min
P3	An employee cannot work and there is no alternative	PC without network in VAL-01 (faulty card)	2 h
P4	Inconvenience with an alternative	a printer (there are others), the computer is slow	1 business day

3. Language models

The four models meet the same contract, `generate(system, messages) → text`, so the rest of the system does not know which one it is using and the rules apply equally to all of them.

Model	What it is	What it is used for	What it needs
ScriptedModel	Responses fixed in advance	The 341 tests and the evaluation	Nothing
SimulatedModel	Keyword rules, deterministic	Demo without a token and web (default)	Nothing
OllamaModel (qwen2.5:14b)	Real model run on my computer with Ollama	The two real traces of the report	Free: install Ollama and the model (9 GB)
HFModel (Qwen2.5-72B)	Real model on Hugging Face servers	Optional	Token and paid credits

The initial plan was Hugging Face, but it no longer gives free credits: two accounts returned *402 Payment Required*. As the professor indicated in class, I used Ollama, which runs the model locally with no token and no cost. The Hugging Face code is still available with `--model hf`. Neither is needed for grading: the tests and the README use the simulated model.

4. Tools and knowledge

Seven tools in a registry with a **description** and a **reversible** flag; all go through a single entry point (`execute`) that turns any failure into a `ToolError`. The two irreversible ones (restart_service, close_ticket) are never run inside an agent's loop: knowledge only proposes them. The network data are loaded in memory, so every run starts from the same state.

Retrieval is a **hand-written BM25** over 15 documents of my own (11 kb-* articles with Symptoms, Cause and Steps, and 4 policies: service catalog, escalation, restarts and priorities). Each fragment is a section, so citations are precise (`docs/kb-network-01.md#Steps`). I chose it over embeddings because it needs no network or dependencies, and it is deterministic and explainable. Limitation: it does not understand synonyms; this is mitigated with employee phrasings in each Symptoms section.

5. Robustness and safety

Rule (in code)	What it guarantees
R1 Confirmation	An irreversible action only runs if confirm() returns true. If the person says no, it does not run and the incident is escalated.
R2 Guardrail	A critical service during business hours is only restarted if it is the diagnosed one and is down or degraded; never in another office; at most one restart per service. If it blocks, the user is not even asked.
R3 Step limit	12 steps (model responses and tool calls): prevents loops and caps spending. When exceeded, step_limit_reached.
R4 Errors	Tool errors, disallowed tools and unreadable JSON become observations; two unreadable responses in a row → escalation.
R5 Citations	A citation only counts if search_docs returned that fragment in this run; made-up ones are removed. Without a valid citation there is no resolved.

In addition: the code raises the priority if the data require it (a critical service down is P1 even if the model says P3) and never lowers it; the answer to the employee is written by a template, so the model cannot promise response times. In Claude Code, `settings.json` only allows reading and testing, asks for permission for dependencies, credit-spending runs and commits, and denies reading `.env` (also via the terminal), deleting, pushing and sending data out. The token lives only in my `.env`: it is not in the code or in the submission.

6. Tests and evaluation

341 tests with **model doubles** (ScriptedModel with fixed responses and a fake client for Hugging Face): deterministic, without network and without a token. Each rule R1–R5 has tests that fail if it breaks; this was checked by breaking each rule on purpose (mutations).

The **evaluation** consists of 11 cases whose expected result I defined by applying my rules, without looking at the code (independent oracle). It measures the **result** (final, type, priority) and the **trajectory** (C = classifier, D = diagnosis, K = knowledge, E = escalation). It runs with a script (it checks the rules) and with

the simulated model (what a user without a token would see; cases 9 and 10 need scripted model errors, so they only run with the script).

#	Case	Expected: final · type · priority · trajectory
1	“I have no internet”, MAD-01, confirms (vpn down)	resolved · network · P1 · C-D-K
2	Same, but rejects the restart (R1)	escalated · network · P1 · C-D-K-E
3	“I have no internet”, VAL-01 (broken card): same symptom, different cause	resolved · network · P3 · C-D-K
4	Printer, MAD-02, T-0007 reopened	escalated · printer · P4 · C-D-E
5	ERP down in MAD-02 and BCN-01; the model says P2	escalated · app_access · P1 · C-D-E
6	Email degraded, BCN-01; the model says P4	resolved · email · P2 · C-D-K
7	“Can you recommend a restaurant?”	rejected · out_of_scope · — · C
8	Nonexistent office (SEV-01)	rejected · — · — · no agents
9	Citation made up by the model (R5)	escalated · network · P1 · C-D-K-E
10	Proposes restarting the ERP with the vpn diagnosed (R2)	escalated · network · P1 · C-D-K-E
11	“My computer is slow” (my own case)	resolved · hardware · P4 · C-D-K

First pass: 9/11 with the script and 6/9 with the simulated model. I did not change the expected results; I analysed each failure. One was in the code (a rejection kept the classifier's P4 priority: what is not processed should have no priority), one was content (there was no article on slow computers) and three were in the simulated model (it did not recognise out-of-scope requests or “computer”, and always started at P3). Case 11 with the simulated model **got the final right by chance**: “resolved” citing the network-card article. That is why the trajectory and the data are also evaluated. After the fixes: **11/11 and 9/9**.

7. Two real traces

With Ollama (section 3, `--model ollama`). The first traces, produced with the Spanish version of the system and **qwen2.5:7b**, ended escalated: the model returned JSON outside the contract (R4), did not call tools and made up citations (R5 removed them). The rules prevented any harm, but the prompt did not state the exact format, the allowed values or what each service is. I added them (generated from the same lists the code validates) and switched to **qwen2.5:14b**; with that, the Spanish trace 1 ended resolved after I confirmed the vpn restart. After translating the system to English, I re-ran both traces with qwen2.5:14b (temperature 0, 19 October 2026). They are shown exactly as `--trace` prints them: one inside the domain and one outside it, as the assignment requires.

Trace 1. “I have no internet” in MAD-01, ticket T-0001: escalated with P1; the restart is never proposed.

```
Final: escalated
Type: network · Priority: P1
Escalation: ESC-0001

Answer to the employee:
Your incident has been escalated to second level with the ID ESC-0001 (priority P1). First response time:
      15 minutes.

Trace:
   1. [classifier] response -> {"type": "network", "priority": "P2", "reason": "An employee cannot access
      the internet, which is necessary for their work, but it's not clear if it's a critica…
   2. [diagnosis] response -> {"tool": "check_service", "arguments": {"service": "vpn", "office": "MAD-
      01"}}
   3. [diagnosis] check_service({"service": "vpn", "office": "MAD-01"}) -> {"service": "vpn", "office":
      "MAD-01", "status": "down"}
   4. [diagnosis] response -> {"cause": "The Office VPN tunnel to the data center is down.", "service":
      "vpn", "evidence": ["The Office VPN tunnel to the data center is down.", "check_servic…
   5. [orchestrator] note -> priority raised from P2 to P1: service vpn affected in MAD-01
   6. [knowledge] response -> {"procedure": "Restart the Office VPN tunnel to the data center.",
      "citations": ["docs/network_services.md#vpn-tunnel-restart"], "action": {"tool": "restart_ser…
   7. [orchestrator] note -> citations removed because they were not retrieved: docs/network_services.
      md#vpn-tunnel-restart
   8. [orchestrator] note -> no procedure with valid citations: escalating
   9. [escalation] response -> {"summary": "The Office VPN tunnel to the data center is down, causing a
      loss of internet connectivity in MAD-01.", "tried": ["Restart the Office VPN tunnel to …
  10. [escalation] create_escalation({"ticket": "T-0001", "summary": "The Office VPN tunnel to the data
      center is down, causing a loss of internet connectivity in MAD-01.", "priority": "P1"}) -> {
```

```
"escalation": "ESC-0001", "ticket": "T-0001", "priority": "P1"}
```

Trace 1 does not match my case 1. Diagnosis checks the vpn (down) and the code raises the priority from P2 to P1. The knowledge agent, however, skips search_docs and cites a document it never retrieved (`docs/network_services.md#vpn-tunnel-restart`, which does not exist): R5 removes the citation, without a cited procedure there is no "resolved", and the code escalates with P1 (ESC-0001). The restart never reaches the user, so R1 is not exercised. The code-level rules worked; what failed is the prompt instruction "your first turn is ALWAYS a call to search_docs", which is not enough with this model in English. Future work: enforce that rule in code.

Trace 2. Outside the domain, "Can you recommend a restaurant for lunch near the office?": rejected without going through more agents and without priority (the model proposed P4), like my case 7.

```
Final: rejected
Type: out_of_scope · Priority: -

Answer to the employee:
The request will not be processed: the request is not an office support incident.

Trace:
   1. [classifier] response -> {"type": "out_of_scope", "priority": "P4", "reason": "The request is for
      restaurant recommendation, which is not related to office technical support."}
   2. [orchestrator] note -> rejected: the request is not an office support incident
```

8. Way of working and limitations

I directed Claude Code with a **CLAUDE.md** of fewer than 200 lines (contracts, rules and closed values), least-privilege permissions and **one command per milestone** with checkable criteria (milestones 0 to 9). The decisions are mine; Claude Code drafted from them and generated the code, which I reviewed. `worklog.md` records decisions, problems, checks and who drafted each thing.

- The simulated model works by keywords: a sentence with none it knows may be misclassified. With the real model, the classifier decides.
- R5 checks that a citation was retrieved, not that it is relevant; and nothing in code forces knowledge to search before answering (trace 1).
- BM25 does not understand synonyms; with more documents a hybrid search would help.
- With a small real model, the escalation agent may list in "tried" things nobody tried (trace 1: the vpn restart); it is not dangerous, but it is inaccurate.
- The 12-step limit was enough with the real model, with little margin: trace 1 used 7 steps, but another real run (ERP in MAD-02, `report/trace_erp_domain.txt`) used all 12 before escalating.

Reproduce: `uv sync` · `uv run pytest` · README commands (simulated model by default, no token). Web console with the agents working live (React): `uv run office-support-web`.